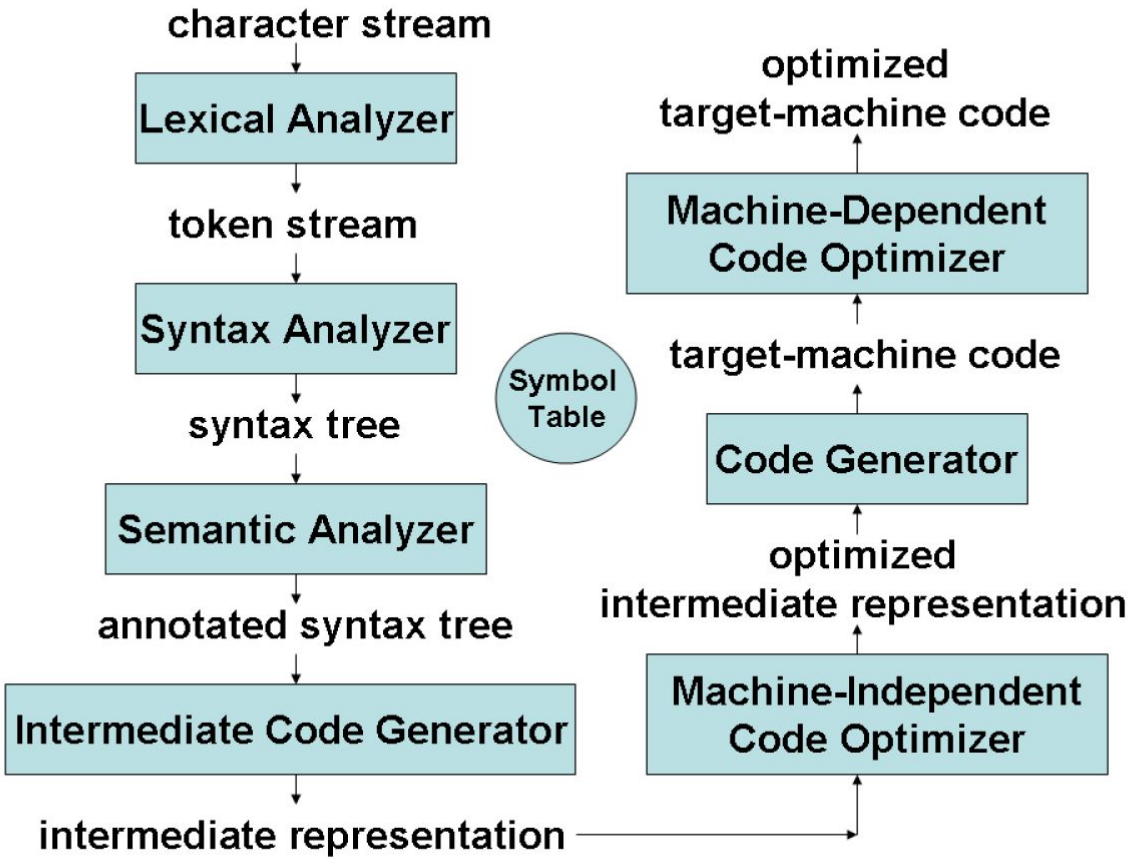
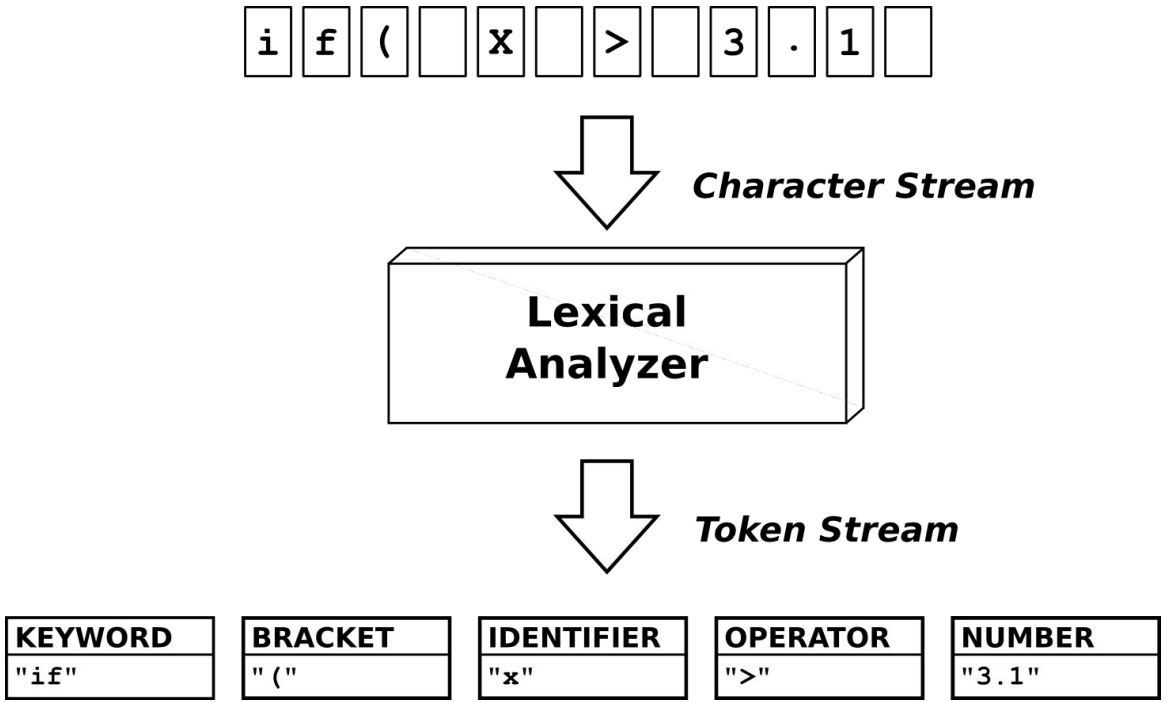
LEX

Lab Session

Compiler Overview



Lexical Analyzer



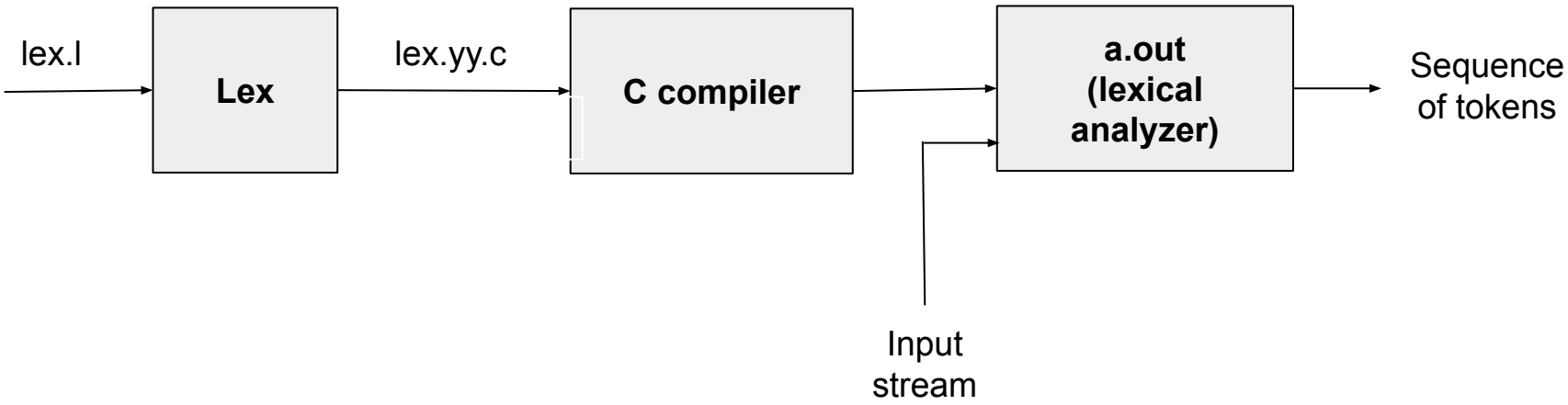
LEX - A Lexical Analyzer Generator

- A tool that automatically generates a lexical analyzer
- LEX has a language for describing regular expressions

```
sudo apt update

sudo apt install flex
```

Lex Pipeline



General structure of a LEX program

{definitions} *-> Optional*

%% *-> indicates the beginning of the rules*

{rules} *-> Required in general* ⟶ regexp { action(s) }

%% *-> Optional*

{user subroutines} *-> Optional*

LEX Example

```
%%
[A-Z]+    {ECHO; printf("\n");}
.|\n       ;
%%
yywrap(){}
main(){yylex();}

/* Input */                    /* Output */
wewevWEUFWIGhHkkH               WEUFWIG
sdcwehSDWEhTkFLksewT            H
                                H
                                SDWE
                                T
                                FL
                                T
```

How to run?

- `flex eg.l`

  `#generates lex.yy.c file`
- `gcc lex.yy.c`
- `./a.out`

Definition section

- Definitions Section contains definitions and included code
  - Definitions are like macros and have the following form: **name translation**

```
digit [0-9]
number {digit} {digit}*
```

  - Included code is all code included between **%{** and **%}**

```
%{
    float number; int count=0;
%}
```

Action section

- Contains patterns and actions
- regular_expression { // C code}
- For example,

```
[0-9]+          { printf("NUMBER: %s\n", yytext); }
[a-zA-Z]+       { printf("WORD: %s\n", yytext); word_count++; }
.               { printf("OTHER: %s\n", yytext); }
```

User subroutines section

- It defines the main function and helper functions
- For example,

```
%%
int main()
{
    yylex();   // call the lexer
    return 0;
}

int yywrap() {
        // called by yylex on reaching EOF
        // returns 1 by default - can be redefined if necessary
}
```

Language aspects handled by lexical analyzer

- Indentation for scoping?
- lexical errors
  - Unrecognized symbol
  - Unmatched comments/inverted commas
- Keywords recognition
- Operators & other legal symbols
- Rules for identifier format
- Case sensitivity
- Dealing with ambiguity such as ++ and +

Resolving Ambiguity

- When more than one expression can match the current input

1) The longest match is preferred.

2) Among rules which matched the same number of characters,

   the rule given first is preferred.

Matched strings

- **yytext** : character array
- contains the actual matched text by a regex rule
  - [a−z]+ printf("%s", yytext);
  - [a−z]+ ECHO;

Predefined Global Variables

- yyin (FILE*) - stdin by default
- yyout (FILE*) - stdout by default
- yyleng : int // number of characters matched
- yylineno : int

LEX Examples

Lexer for a subset of SQL

- Scan SELECT statements
- `SELECT name, age FROM customers WHERE age > 30;`

Example programs to try

- Find and replace a particular string

  (eg. replace “foo” with “bar”)

- Markdown → HTML conversion

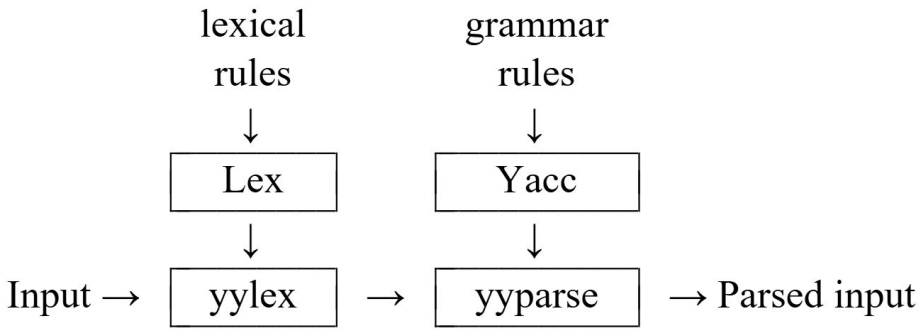
  (eg. **bold** → bold)

YACC

Lab Session

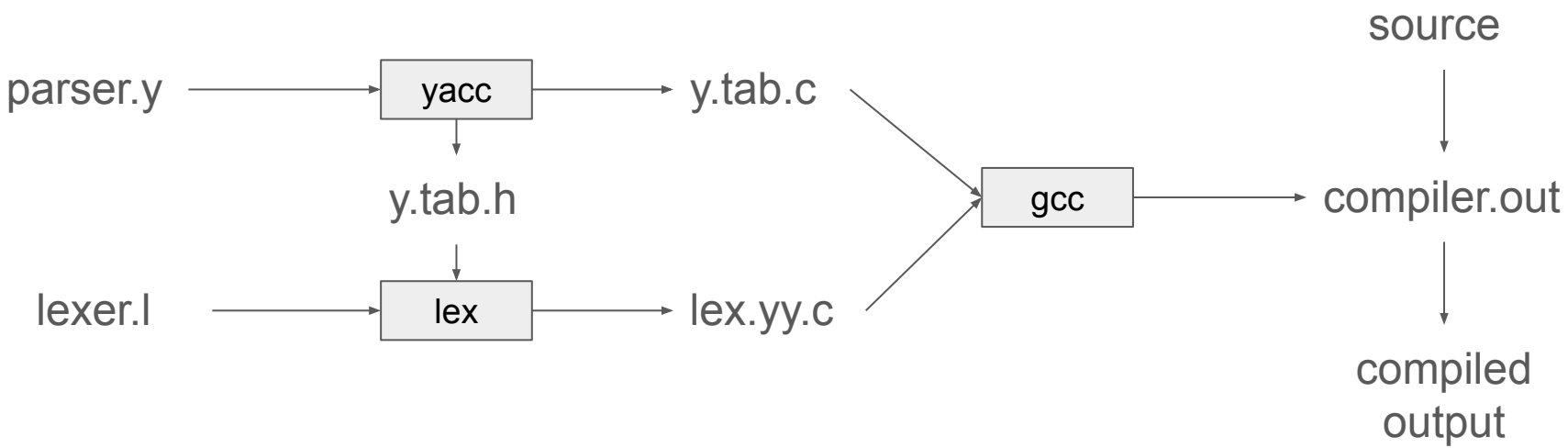
9 Sept 2026

Lex with Yacc



Source: Lex Manual

Building a compiler



Yacc Program Structure

{declarations} *-> Optional*

%% *-> Indicates the beginning of the rules*

{production rules} *-> Required* Non-Terminal : Body {action(s)};

%% *-> Optional*

{programs} *-> Optional*

Declaration Section

%token name1 name2 . . .

name1, name2 here - tokens / terminals

Names not in declaration section - assumed to be a non-terminal

%start symbol

(symbol here is a non-terminal)

Calc 1 example

$$
\begin{array}{rcl}
expr & \rightarrow & expr + term \mid expr - term \mid term \\
term & \rightarrow & term * factor \mid term / factor \mid factor \\
factor & \rightarrow & \textbf{digit} \mid (expr)
\end{array}
$$

Try these

1. Accept integers
   a. -1 + 3 = 2
2. Parenthesized expressions
   a. 2 * (1 + 2) = 6)
   b. 2 * 1 + 2 = 4

Input:

x = 1

y = 1

x + y

Output:

2

Shift Reduce Conflicts

Calc_SR example

Removing ambiguity among operators

Precedence:
Determines order of operations between operators of different priority

Eg: a+b*c
(a+b)*c OR a+(b*c) ?

Associativity:
Determines order of operations between operators of same priority/precedence

Eg: a-b-c
(a-b)-c OR a-(b-c) ?

Associativity in Yacc

%left ‘+’

1 + 2 + 3 solved as (1 + 2) + 3

%right ‘+’

1 + 2 + 3 solved as 1 + (2 + 3)

%nonassoc ‘+’

1 + 2 + 3 is considered illegal and a syntax error!

Try these

Input:

x = 1

y = 1

x + y

Output:

2

Lex and YACC - further practice

https://github.com/sreevyshnavi02/lex_yacc_lab_session_sept26

Custom Data Types for Attributes

We can redefine YYSTYPE to a union containing types that we intend to use in attributes.

```
typedef union{
  bool positive;
  float value;
} YYSTYPE;

term : sign magnitude    {
    $$.value = $2.value;
    if(!$1.positive) $$.value = -$$.value ;
}  ;
```

Union allows multiple data types to be stored in the same memory location. (Only one type can be used at a time).

```
typedef union {
    bool  positive;
    float value;
    char *str;
} YYSTYPE;
```

```
%token <positive> SIGN
%token <value>    NUMBER
%token <str>      STRING
%type  <value>    term
```

Example: string attributes

Example: adding strings as attributes

```
str : str + str    {
    // allocate space for new string
    int len1 = strlen($1.str);
    int len3 = strlen($3.str);
    $$.str = (char*) malloc(len1+len3+1);

    // concatenate attributes from both children
    strcpy($$.str +    0, $1.str);
    strcpy($$.str + len1, $3.str);
    free($1.str);
    free($3.str);
} ;
```

Left vs Right Recursion

Take the example input:
s ; s ; s

- Right Recursion: reads right of ‘;’ to gather information about the following variable ‘program’.
- It reads the entire input before reducing.
- Order of actions: {a2} {a2} {a2} {a1} {a1} {a1}

Right Recursion

```
program : statement                 {a1}
        | statement ';' program     {a2} ;
```

Left vs Right Recursion

Take the example input:
s ; s ; s

- Left Recursion: It knows the ‘program’ variable in the RHS ends with a ‘;’.
- Does not read the rest of the program to decide the reduction.
- Order of actions: {a2} {a1} {a2} {a1} {a2} {a1}

Left Recursion

```
program : statement                {a1}
    | program ';' statement        {a2} ;
```

LLVM: Kaleidoscope

Lab session

Reference: https://llvm.org/docs/tutorial/

07 Oct, 2026

Kaleidoscope DSL

Data types: Only 64-bit floating point type

```
# Compute the x'th fibonacci number.
def fib(x)
  if x < 3 then
    1
  else
    fib(x-1)+fib(x-2)

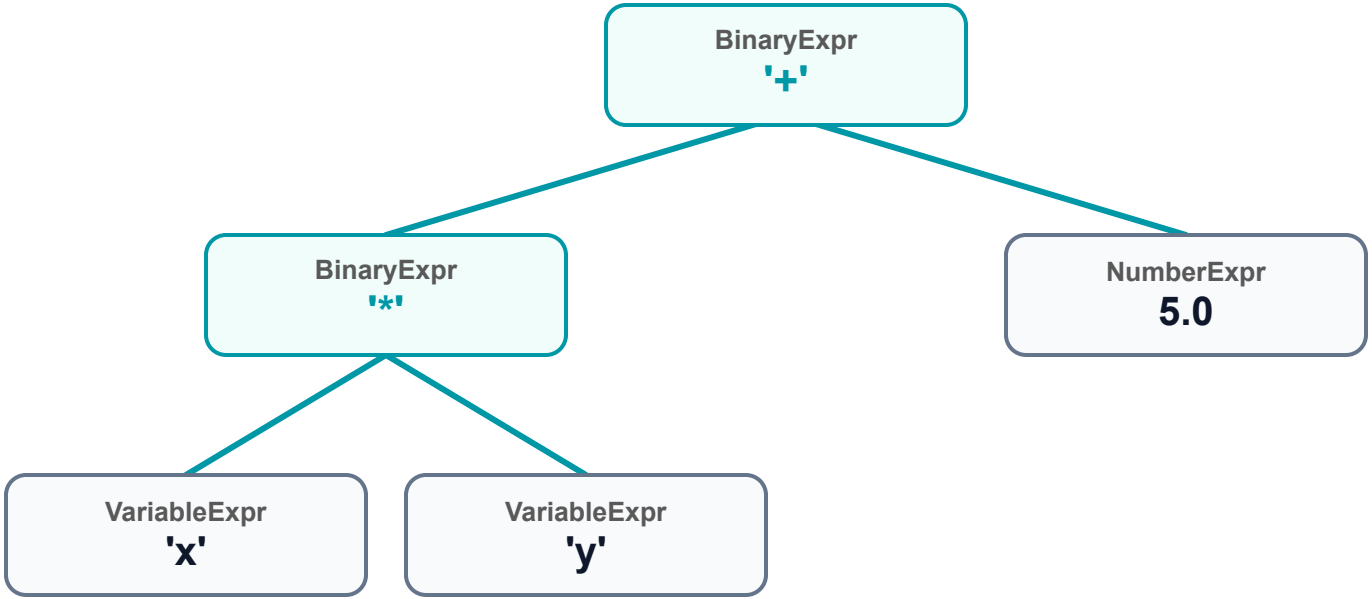
# This expression will compute the 40th number.
fib(40)
```

```
extern sin(arg);
extern cos(arg);
extern atan2(arg1 arg2);

atan2(sin(.4), cos(42))
```

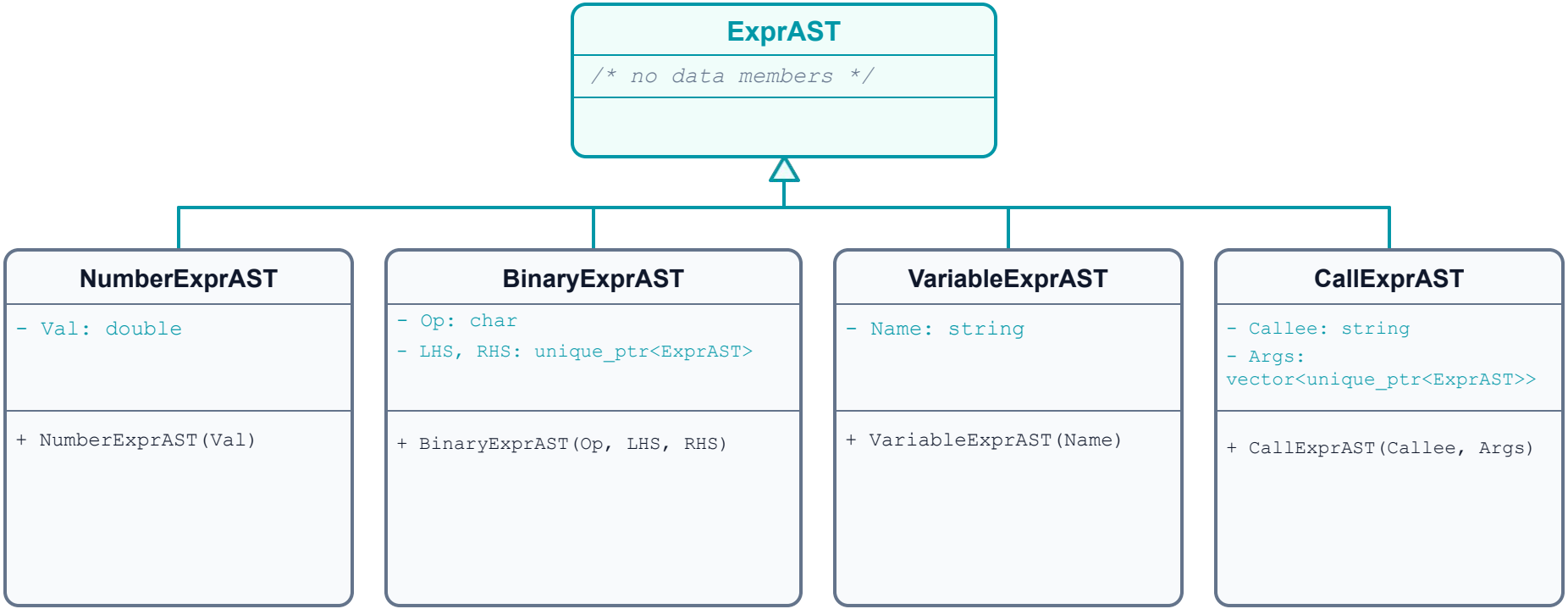
Abstract Syntax Tree

Expression: `(x * y) + 5.0`

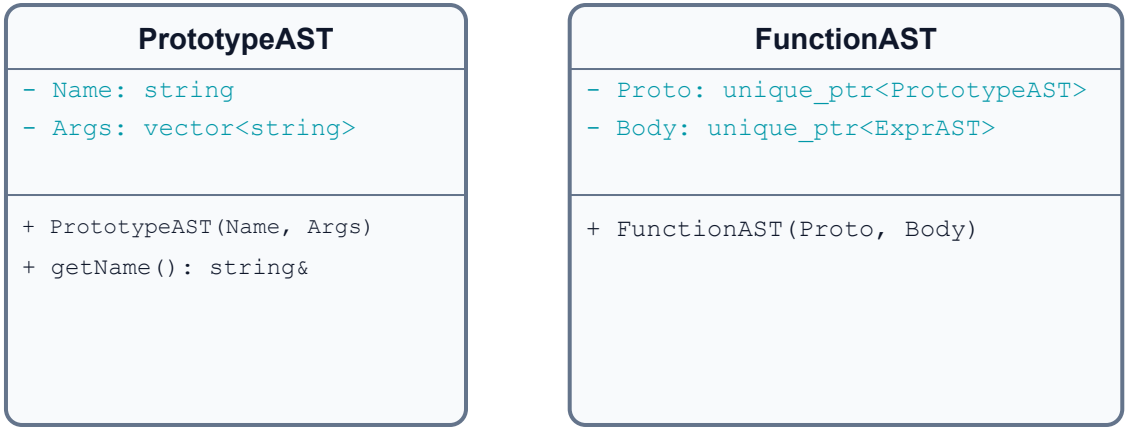


AST Class Hierarchy

Base class **`ExprAST`** and derived AST expression nodes

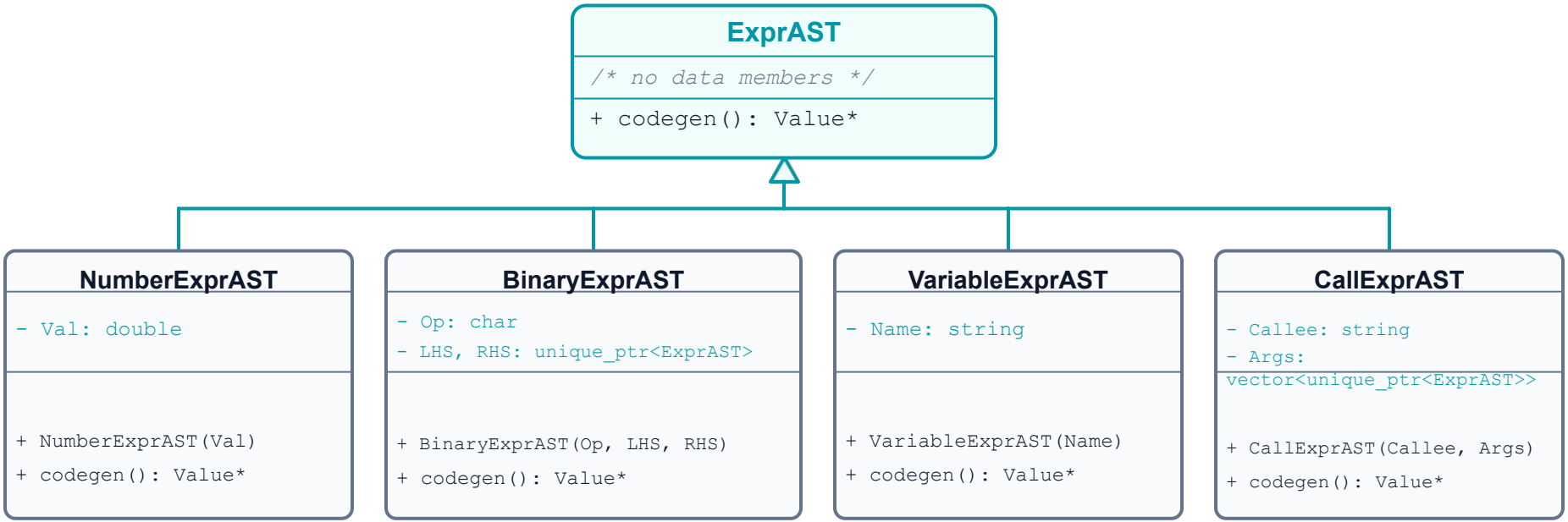


AST Class Hierarchy



Code Generation

codegen() - to emit IR for that AST node along with all the things it depends on



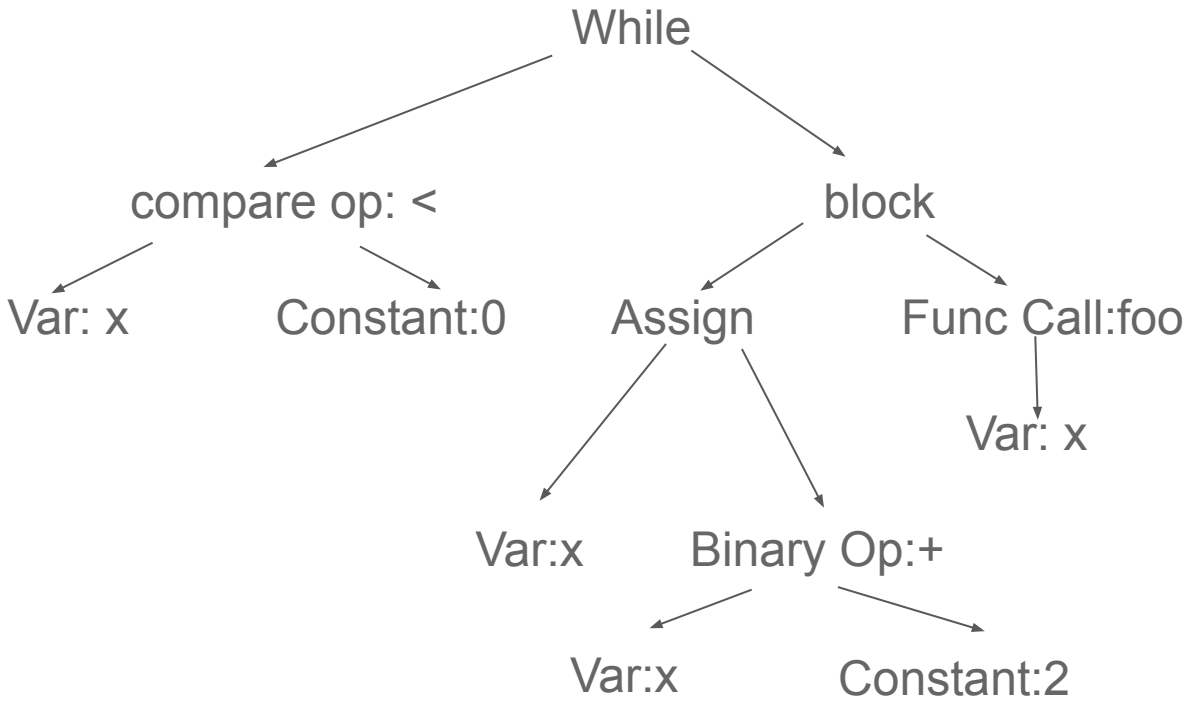
Code Generation

Abstract Syntax Tree

- A tree representation of the source code
- Conveys the abstract syntactic structure of the source code
- Each node in the tree represents a construct occurring in the source code

Example

```
while x < 0 :
    x = x+2
    foo(x)
```



Clang AST

- Closely Resembles the source code.
- Nodes
  - Statement
    - IfStmt, ReturnStmt , GotoStmt
  - Declaration
    - FunctionDecl, VarDecl, ParmVarDecl
  - Expressions
    - BinaryOperator, IntegerLiteral
- Each AST node is represented by a C++ Class structure with associated member functions and member variables

Clang AST

```
1  void test() {
2      int a;
3      int b;
4
5      a =10;
6
7      if(a<10){
8
9          a--;
10     }
11     else{
12
13         a++;
14     }
15
16 }
```

```
(base) cs23mtech12010@kodanda:/data/cs23mtech12010/compilers2/llvm-project/build$ ./bin/clang -Xclang -ast-dum
ter=test -fsyntax-only '/data/cs23mtech12010/compilers2/test.cpp'
Dumping test:
FunctionDecl 0x56270171f468 </data/cs23mtech12010/compilers2/test.cpp:1:1, line:16:1> line:1:6 test 'void ()'
`-CompoundStmt 0x56270171f830 <col:13, line:16:1>
  |-DeclStmt 0x56270171f5d8 <line:2:5, col:10>
  | `-VarDecl 0x56270171f570 <col:5, col:9> col:9 used a 'int'
  |-DeclStmt 0x56270171f670 <line:3:5, col:10>
  | `-VarDecl 0x56270171f608 <col:5, col:9> col:9 b 'int'
  |-BinaryOperator 0x56270171f6c8 <line:5:5, col:8> 'int' lvalue '='
  | |-DeclRefExpr 0x56270171f688 <col:5> 'int' lvalue Var 0x56270171f570 'a' 'int'
  | `-IntegerLiteral 0x56270171f6a8 <col:8> 'int' 10
  `-IfStmt 0x56270171f800 <line:7:5, line:14:5> has_else
    |-BinaryOperator 0x56270171f740 <line:7:8, col:10> 'bool' '<'
    | |-ImplicitCastExpr 0x56270171f728 <col:8> 'int' <LValueToRValue>
    | | `-DeclRefExpr 0x56270171f6e8 <col:8> 'int' lvalue Var 0x56270171f570 'a' 'int'
    | `-IntegerLiteral 0x56270171f708 <col:10> 'int' 10
    |-CompoundStmt 0x56270171f798 <col:13, line:10:5>
    | `-UnaryOperator 0x56270171f780 <line:9:9, col:10> 'int' postfix '--'
    |   `-DeclRefExpr 0x56270171f760 <col:9> 'int' lvalue Var 0x56270171f570 'a' 'int'
    `-CompoundStmt 0x56270171f7e8 <line:11:9, line:14:5>
      `-UnaryOperator 0x56270171f7d0 <line:13:9, col:10> 'int' postfix '++'
        `-DeclRefExpr 0x56270171f7b0 <col:9> 'int' lvalue Var 0x56270171f570 'a' 'int'
```

Node Structure of Clang AST

<Nodetype> <in-memory address> <source location> <meta-data>



Example: BinaryOperator 0x40b0808 <line:5, col:9> 'int' '='

Dumping Clang AST

- ./bin/clang -Xclang -ast-dump -fsyntax-only 'test.cpp'

Print AST for specific function

- ./bin/clang -Xclang -ast-dump -Xclang **-ast-dump-filter=test** -fsyntax-only 'test.cpp'

Traversing AST: AST Matchers

- Describe and match specific patterns in AST
- Traverses the AST in Preorder
- Clang provides Three types of Matchers
  - Node Matchers
  - Narrowing Matchers
  - Traversal Matchers

Examples

CASE 1

```
for( int i =0; i < 10 ; i++){

}
```

CASE 2

```
for( int i =20; i < 10 ; i++){

}
```

```
forstmt(
    hasLoopInit(
        declStmt(
            hasSingleDecl(
                varDecl(
                    hasInitializer(
                        integerLiteral(equals(0)))))));
```

Implementing AST Matcher

```
DeclarationMatcher FunctionDeclMatcher =
functionDecl(hasDescendant(varDecl().bind("variables"))).bind("function");
```

```
class Simple_Pass_Handler : public MatchFinder::MatchCallback {
public:
    int count =0;
    int Var_count =0;
  virtual void run(const MatchFinder::MatchResult &Result) override {
    if (const FunctionDecl *FD = Result.Nodes.getNodeAs<FunctionDecl>("function")) {
        count++;
      outs() << "Function name: " << FD->getNameAsString() <<" "<< count<<"\n";
    }
  }
};
```

Implementing AST Matcher

```
int main(int argc, const char **argv) {
  auto ExpectedParser = CommonOptionsParser::create(argc, argv, Simple_Pass);

  if (!ExpectedParser) {
    // Fail gracefully for unsupported options.
    errs() << ExpectedParser.takeError();
    return 1;
  }

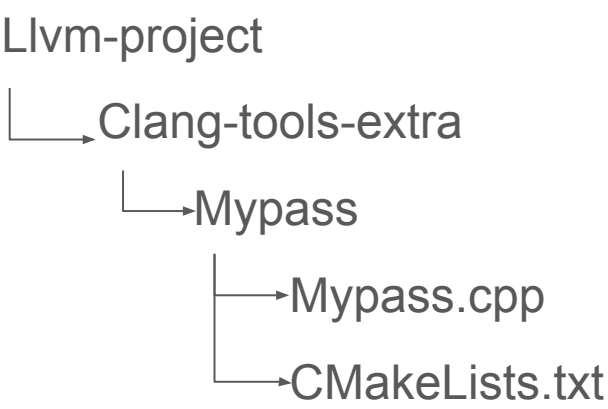
  CommonOptionsParser &OptionsParser = ExpectedParser.get();
  ClangTool Tool(OptionsParser.getCompilations(),
                 OptionsParser.getSourcePathList());

  Simple_Pass_Handler Handler;
  MatchFinder Finder;
  Finder.addMatcher(FunctionDeclMatcher, &Handler);

  Tool.run(newFrontendActionFactory(&Finder).get());

  outs() << "The Total VarCount " << Handler.Var_count <<"\n";
}
```

Directory Structure



Question 1

- Write a matcher to print names of all the functions defined

Question 2

- Write a matcher to dump all the loops in a function

Question 3

- Match all the variable declarations that are not parameters